Database Schema Document

Logistics Reverse Auction & Transporter Bidding Platform

Prepared by: Anshuman Tiwari **Companion to:** PRD_Logistics_Reverse_Auction_App.md, Tech_Stack_Logistics_Reverse_Auction_App.md **Database:** Neon Postgres **Scope:** Tables are tagged `[MVP]` or `[FULL VISION]` — build only the MVP tables this weekend; the rest are documented for completeness and to guide future migrations.

---

1. Entity Relationship Overview

```
users (1) ──< company_profiles (1)
users (1) ──< transporter_profiles (1)
users [COMPANY] (1) ──< tenders (many)
tenders (1) ──< bids (many) >── users [TRANSPORTER]
tenders (1) ──< shortlists (many, max 5) >── users [TRANSPORTER]
tenders (1) ──< negotiation_rounds (many) >── users [TRANSPORTER]
tenders (1) ──1 competitive_statements
tenders (1) ──1 finalizations
users (1) ──< notifications (many)

[FULL VISION only]
subscription_plans (1) ──< user_subscriptions (many) >── users
user_subscriptions (1) ──< payments (many)
users [ADMIN/SUPER_ADMIN] (1) ──< audit_logs (many)
```

Cardinality notes:

- One `tender` has many `bids` (one per transporter, revisable — see §3.4 on bid revision handling).
- One `tender` has exactly 5 `shortlists` rows once bidding closes (L1–L5).
- One `tender` has one `competitive_statements` row and one `finalizations` row, created at the end of the flow.

---

2. Enums

sql

```
CREATE TYPE user_role AS ENUM ('COMPANY', 'TRANSPORTER', 'ADMIN', 'SUPER_ADMIN'

CREATE TYPE tender_status AS ENUM (
  'DRAFT', 'PUBLISHED', 'BIDDING_CLOSED', 'SHORTLISTED',
  'NEGOTIATION', 'AWARDED', 'CANCELLED', 'CLOSED'
);

CREATE TYPE kyc_status AS ENUM ('PENDING', 'VERIFIED', 'REJECTED');

CREATE TYPE finalization_status AS ENUM ('PENDING', 'TRANSPORTER_CONFIRMED', 'C

CREATE TYPE notification_type AS ENUM (
  'NEW_TENDER', 'OUTBID', 'SHORTLISTED', 'NOT_SHORTLISTED',
  'NEGOTIATION_OPEN', 'FINALIZATION_REQUEST', 'TENDER_AWARDED', 'TENDER_CLOSED'
);
```

3. Core Tables — [MVP]

3.1 users

Column	Type	Constraints
id	BIGSERIAL	PRIMARY KEY
name	VARCHAR(150)	NOT NULL
email	VARCHAR(150)	UNIQUE, NOT NULL
phone	VARCHAR(20)	UNIQUE
password_hash	VARCHAR(255)	NOT NULL
role	user_role	NOT NULL
is_subscribed	BOOLEAN	DEFAULT FALSE — *mocked subscription flag, see §5*
plan_name	VARCHAR(50)	NULLABLE — *mocked plan label*
fcm_token	TEXT	NULLABLE — device token for push notifications
created_at	TIMESTAMP	DEFAULT now()
updated_at	TIMESTAMP	DEFAULT now()

3.2 `company_profiles`

Column	Type	Constraints
id	BIGSERIAL	PRIMARY KEY
user_id	BIGINT	FK → users(id), UNIQUE, NOT NULL
company_name	VARCHAR(200)	NOT NULL
gst_no	VARCHAR(20)	NULLABLE
address	TEXT	NULLABLE

3.3 `transporter_profiles`

Column	Type	Constraints
id	BIGSERIAL	PRIMARY KEY
user_id	BIGINT	FK → users(id), UNIQUE, NOT NULL
fleet_details	TEXT	NULLABLE — free text or JSON for MVP
service_routes	TEXT	NULLABLE — free text or JSON for MVP
kyc_status	kyc_status	DEFAULT 'PENDING'

3.4 `tenders`

Column	Type	Constraints
id	BIGSERIAL	PRIMARY KEY
company_id	BIGINT	FK → users(id), NOT NULL
origin	VARCHAR(200)	NOT NULL
destination	VARCHAR(200)	NOT NULL
vehicle_type	VARCHAR(100)	NOT NULL
load_details	TEXT	NULLABLE
weight_kg	NUMERIC(10,2)	NULLABLE
bidding_start	TIMESTAMP	NOT NULL
bidding_end	TIMESTAMP	NOT NULL
reserve_price	NUMERIC(12,2)	NULLABLE
negotiation_margin_percent	NUMERIC(5,2)	DEFAULT 10.00
status	tender_status	DEFAULT 'DRAFT'
created_at	TIMESTAMP	DEFAULT now()
updated_at	TIMESTAMP	DEFAULT now()

3.5 `bids`

Column	Type	Constraints
id	BIGSERIAL	PRIMARY KEY
tender_id	BIGINT	FK → tenders(id), NOT NULL
transporter_id	BIGINT	FK → users(id), NOT NULL
price	NUMERIC(12,2)	NOT NULL
remarks	TEXT	NULLABLE
submitted_at	TIMESTAMP	DEFAULT now()
is_active	BOOLEAN	DEFAULT TRUE

> **Bid revision handling (MVP-simple approach):** Rather than updating a row in place (which loses history), insert a new row per revision and mark the previous one `is_active = false`. The "current" bid for ranking purposes is always the latest active row per transporter per tender. This keeps a full audit trail without extra tables.

3.6 `shortlists`

Column	Type	Constraints
id	BIGSERIAL	PRIMARY KEY
tender_id	BIGINT	FK → tenders(id), NOT NULL
transporter_id	BIGINT	FK → users(id), NOT NULL
rank	SMALLINT	NOT NULL — 1 to 5 (L1–L5)
created_at	TIMESTAMP	DEFAULT now()

`UNIQUE (tender_id, transporter_id)`, `UNIQUE (tender_id, rank)`

3.7 `negotiation_rounds`

Column	Type	Constraints
id	BIGSERIAL	PRIMARY KEY
tender_id	BIGINT	FK → tenders(id), NOT NULL
transporter_id	BIGINT	FK → users(id), NOT NULL
offer_price	NUMERIC(12,2)	NOT NULL
submitted_at	TIMESTAMP	DEFAULT now()

3.8 `competitive_statements`

Column	Type	Constraints
id	BIGSERIAL	PRIMARY KEY
tender_id	BIGINT	FK → tenders(id), UNIQUE, NOT NULL
final_l1_transporter_id	BIGINT	FK → users(id), NOT NULL
final_price	NUMERIC(12,2)	NOT NULL
statement_data	JSONB	NOT NULL — snapshot of all bids + negotiation rounds at generation time
generated_at	TIMESTAMP	DEFAULT now()

> Storing the full snapshot as `statement_data JSONB` (rather than re-joining live tables later) guarantees the statement stays accurate even if bid rows change afterward — and it's the fastest way to build the "export" view for MVP without a PDF library.

3.9 `finalizations`

Column	Type	Constraints
id	BIGSERIAL	PRIMARY KEY
tender_id	BIGINT	FK → tenders(id), UNIQUE, NOT NULL
transporter_confirmed_at	TIMESTAMP	NULLABLE
company_confirmed_at	TIMESTAMP	NULLABLE
status	finalization_status	DEFAULT 'PENDING'

3.10 `notifications`

Column	Type	Constraints
id	BIGSERIAL	PRIMARY KEY
user_id	BIGINT	FK → users(id), NOT NULL
type	notification_type	NOT NULL
title	VARCHAR(200)	NOT NULL
body	TEXT	NULLABLE
is_read	BOOLEAN	DEFAULT FALSE
created_at	TIMESTAMP	DEFAULT now()

4. Deferred Tables — `[FULL VISION, not required for MVP]`

These are documented so the schema story is complete in your PRD, but **do not build these for Sunday** — mock subscription state on `users` instead (see §5).

4.1 `subscription_plans`

Column	Type
id	BIGSERIAL PK
role_type	user_role
name	VARCHAR(100)
price	NUMERIC(10,2)
tender_limit	INTEGER
bid_limit	INTEGER
features_json	JSONB

4.2 `user_subscriptions`

Column	Type
id	BIGSERIAL PK
user_id	FK → users(id)
plan_id	FK → subscription_plans(id)
start_date	TIMESTAMP
end_date	TIMESTAMP
payment_status	VARCHAR(30)

4.3 `payments`

Column	Type
id	BIGSERIAL PK
user_id	FK → users(id)
subscription_id	FK → user_subscriptions(id)
amount	NUMERIC(10,2)
gateway_txn_id	VARCHAR(100)
status	VARCHAR(30)

4.4 `audit_logs`

Column	Type
id	BIGSERIAL PK
actor_id	FK → users(id)
action	VARCHAR(100)
entity_type	VARCHAR(50)
entity_id	BIGINT
before_state	JSONB
after_state	JSONB
created_at	TIMESTAMP

---

5. Mocking Subscriptions for MVP

Instead of `subscription_plans` + `user_subscriptions` + `payments`, MVP uses two columns already on `users`:

- `is_subscribed BOOLEAN`
- `plan_name VARCHAR(50)`

`POST /subscriptions/subscribe` simply sets these two fields. This preserves the API shape your Flutter screens need without building the full billing schema.

---

6. Indexes

```sql
CREATE INDEX idx_tenders_status ON tenders(status);
CREATE INDEX idx_tenders_company_id ON tenders(company_id);
CREATE INDEX idx_bids_tender_id ON bids(tender_id);
CREATE INDEX idx_bids_transporter_id ON bids(transporter_id);
CREATE INDEX idx_bids_tender_active ON bids(tender_id, is_active);
CREATE INDEX idx_shortlists_tender_id ON shortlists(tender_id);
CREATE INDEX idx_negotiation_tender_id ON negotiation_rounds(tender_id);
CREATE INDEX idx_notifications_user_unread ON notifications(user_id, is_read);
```

These cover the query patterns you'll hit most: filtering tenders by status, pulling all bids for a tender's leaderboard, and fetching a user's unread notifications.

---

7. Full DDL Script (MVP Tables Only)

Run this directly against your Neon Postgres database, or hand it to your LLM as the ground-truth schema instead of relying on `spring.jpa.hibernate.ddl-auto=update` if you want more predictable table creation:

sql

```
CREATE TYPE user_role AS ENUM ('COMPANY', 'TRANSPORTER', 'ADMIN', 'SUPER_ADMIN'
CREATE TYPE tender_status AS ENUM ('DRAFT', 'PUBLISHED', 'BIDDING_CLOSED', 'SHO
CREATE TYPE kyc_status AS ENUM ('PENDING', 'VERIFIED', 'REJECTED');
CREATE TYPE finalization_status AS ENUM ('PENDING', 'TRANSPORTER_CONFIRMED', 'C
CREATE TYPE notification_type AS ENUM ('NEW_TENDER', 'OUTBID', 'SHORTLISTED', '

CREATE TABLE users (
    id BIGSERIAL PRIMARY KEY,
    name VARCHAR(150) NOT NULL,
    email VARCHAR(150) UNIQUE NOT NULL,
    phone VARCHAR(20) UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    role user_role NOT NULL,
    is_subscribed BOOLEAN DEFAULT FALSE,
    plan_name VARCHAR(50),
    fcm_token TEXT,
    created_at TIMESTAMP DEFAULT now(),
    updated_at TIMESTAMP DEFAULT now()
);

CREATE TABLE company_profiles (
    id BIGSERIAL PRIMARY KEY,
    user_id BIGINT UNIQUE NOT NULL REFERENCES users(id),
    company_name VARCHAR(200) NOT NULL,
    gst_no VARCHAR(20),
    address TEXT
);

CREATE TABLE transporter_profiles (
    id BIGSERIAL PRIMARY KEY,
    user_id BIGINT UNIQUE NOT NULL REFERENCES users(id),
    fleet_details TEXT,
    service_routes TEXT,
    kyc_status kyc_status DEFAULT 'PENDING'
);

CREATE TABLE tenders (
    id BIGSERIAL PRIMARY KEY,
    company_id BIGINT NOT NULL REFERENCES users(id),
    origin VARCHAR(200) NOT NULL,
    destination VARCHAR(200) NOT NULL,
    vehicle_type VARCHAR(100) NOT NULL,
    load_details TEXT,
    weight_kg NUMERIC(10,2),
    bidding_start TIMESTAMP NOT NULL,
    bidding_end TIMESTAMP NOT NULL,
```

```
    reserve_price NUMERIC(12,2),
    negotiation_margin_percent NUMERIC(5,2) DEFAULT 10.00,
    status tender_status DEFAULT 'DRAFT',
    created_at TIMESTAMP DEFAULT now(),
    updated_at TIMESTAMP DEFAULT now()
);

CREATE TABLE bids (
    id BIGSERIAL PRIMARY KEY,
    tender_id BIGINT NOT NULL REFERENCES tenders(id),
    transporter_id BIGINT NOT NULL REFERENCES users(id),
    price NUMERIC(12,2) NOT NULL,
    remarks TEXT,
    submitted_at TIMESTAMP DEFAULT now(),
    is_active BOOLEAN DEFAULT TRUE
);

CREATE TABLE shortlists (
    id BIGSERIAL PRIMARY KEY,
    tender_id BIGINT NOT NULL REFERENCES tenders(id),
    transporter_id BIGINT NOT NULL REFERENCES users(id),
    rank SMALLINT NOT NULL,
    created_at TIMESTAMP DEFAULT now(),
    UNIQUE (tender_id, transporter_id),
    UNIQUE (tender_id, rank)
);

CREATE TABLE negotiation_rounds (
    id BIGSERIAL PRIMARY KEY,
    tender_id BIGINT NOT NULL REFERENCES tenders(id),
    transporter_id BIGINT NOT NULL REFERENCES users(id),
    offer_price NUMERIC(12,2) NOT NULL,
    submitted_at TIMESTAMP DEFAULT now()
);

CREATE TABLE competitive_statements (
    id BIGSERIAL PRIMARY KEY,
    tender_id BIGINT UNIQUE NOT NULL REFERENCES tenders(id),
    final_l1_transporter_id BIGINT NOT NULL REFERENCES users(id),
    final_price NUMERIC(12,2) NOT NULL,
    statement_data JSONB NOT NULL,
    generated_at TIMESTAMP DEFAULT now()
);

CREATE TABLE finalizations (
    id BIGSERIAL PRIMARY KEY,
    tender_id BIGINT UNIQUE NOT NULL REFERENCES tenders(id),
```

```
    transporter_confirmed_at TIMESTAMP,
    company_confirmed_at TIMESTAMP,
    status finalization_status DEFAULT 'PENDING'
);

CREATE TABLE notifications (
    id BIGSERIAL PRIMARY KEY,
    user_id BIGINT NOT NULL REFERENCES users(id),
    type notification_type NOT NULL,
    title VARCHAR(200) NOT NULL,
    body TEXT,
    is_read BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMP DEFAULT now()
);

CREATE INDEX idx_tenders_status ON tenders(status);
CREATE INDEX idx_tenders_company_id ON tenders(company_id);
CREATE INDEX idx_bids_tender_id ON bids(tender_id);
CREATE INDEX idx_bids_transporter_id ON bids(transporter_id);
CREATE INDEX idx_bids_tender_active ON bids(tender_id, is_active);
CREATE INDEX idx_shortlists_tender_id ON shortlists(tender_id);
CREATE INDEX idx_negotiation_tender_id ON negotiation_rounds(tender_id);
CREATE INDEX idx_notifications_user_unread ON notifications(user_id, is_read);
```

---

8. Notes for the LLM Generating Your Spring Boot Entities

- Use `@Enumerated(EnumType.STRING)` for all enum columns so Postgres enum values map cleanly to Java enums.
- Use `BigDecimal` (not `double`/`float`) for all `NUMERIC` price/percentage fields to avoid rounding errors in bid comparisons.
- `statement_data` on `competitive_statements` maps to a `JSONB` column — either store it as a raw `String` (simplest for MVP, serialize/deserialize manually) or use `hypersistence-utils` if you want a mapped `Map<String, Object>`/POJO. For a weekend deadline, the raw `String` approach is faster to implement correctly.
- Set `spring.jpa.hibernate.ddl-auto=update` if you want Hibernate to auto-create tables from your `@Entity` classes instead of running this DDL manually — but if you do that, the enum types (`user_role`, `tender_status`, etc.) still need to exist in Postgres beforehand, since Hibernate won't create Postgres native enum types for you. Simplest fix: run the `CREATE TYPE` statements from §7 manually once, then let Hibernate manage the tables.